

Unit-II- Operating System:

Process Concept:

A process is defined as an entity which represents the basic unit of work to be implemented in the system. To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

An operating system executes a variety of programs that run as a process.

Process – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process

Multiple parts

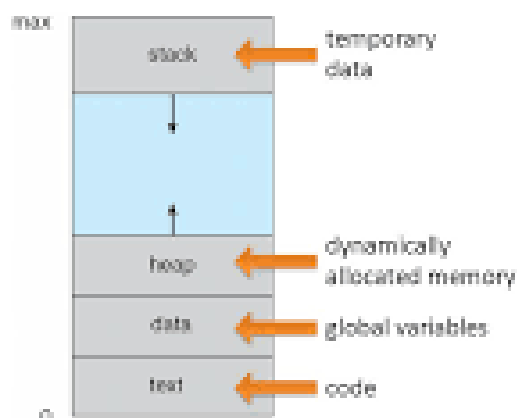
- The program code, also called text section
- Current activity including program counter, processor registers
- Stack containing temporary data
- Function parameters, return addresses, local variables
- Data section containing global variables
- Heap containing memory dynamically allocated during run time
- Program is **passive** entity stored on disk (**executable file**); process is **active**
 - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
 - Consider multiple users executing the same program

Process Concept

A **process** is a *program in execution*.

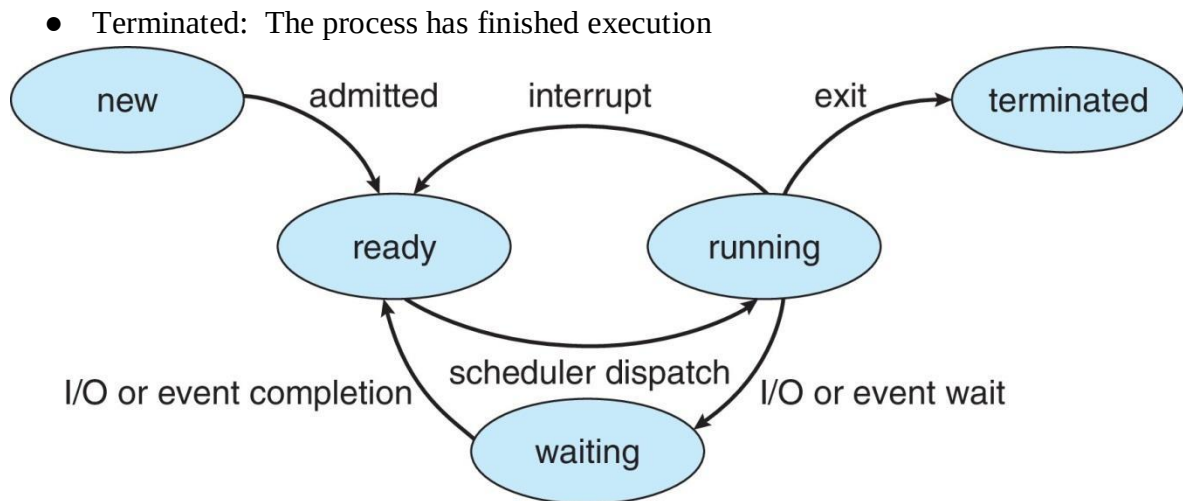
We use the terms *process*, *job*, and *task* interchangeably.

A Process in Memory



As a process executes, it changes state

- New: The process is being created
- Running: Instructions are being executed
- Waiting: The process is waiting for some event to occur
- Ready: The process is waiting to be assigned to a processor



Process Control Block (PCB)

Information associated with each process(also called task control block)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

process state
process number
program counter
registers
memory limits
list of open files
...

Process Scheduling:

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy. Process scheduling is an essential part of a Multiprogramming operating systems.

What is process scheduling and its types?

Process Scheduling Queues

The OS maintains all PCBs in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following important process scheduling queues –

- Job queue – This queue keeps all the processes in the system.
- Ready queue – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- Device queues – The processes which are blocked due to unavailability of an I/O device constitute this queue.

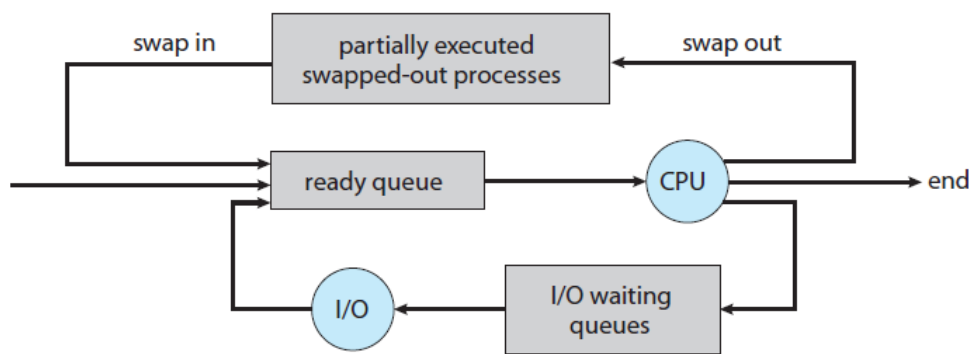


Figure 3.7 Addition of medium-term scheduling to the queueing diagram.

Schedulers

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types –

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Long Term Scheduler

It is also called a job scheduler. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long-term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler. When a process changes the state from new to ready, then there is use of long-term scheduler.

Short Term Scheduler

It is also called as CPU scheduler. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.

Short-term schedulers, also known as dispatchers, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

Medium Term Scheduler

Medium-term scheduling is a part of swapping. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage.

This process is called swapping, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

Comparison among Scheduler

S.N.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

INTERPROCESS COMMUNICATION:

- ❖ A process can be either an independent process or a cooperating process.
- ❖ A process is independent if it cannot affect or be affected by the other processes executing in the system. Any process that does not share data with any other process is independent.
- ❖ A process is cooperating if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.
- ❖ Advantages of cooperating process
 - i) Information sharing
 - ii) Computation speedup
 - iii) Modularity
 - iv) Convenience.
- ❖ DEFINITION: An Interprocess communication is a mechanism that allows the cooperating process to exchange data and communication among each other.
- ❖ There are two fundamental models of Interprocess communication
 - o Shared Memory model
 - o Message passing model
- ❖ In shared memory model a region of memory is shared by the cooperating process. Processes can then exchange information by reading and writing data to the shared region.
- ❖ In the message-passing model, communication takes place by means of messages exchanged between the cooperating processes.
- ❖ Message passing is also easier to implement in a distributed system than shared memory.
- ❖ The shared memory is faster than that of message passing.

Shared-Memory Systems:

- ❖ Interprocess communication using shared memory requires communicating processes to establish a region of shared memory.

- ❖ Shared-memory region resides in the address space of the process creating the shared-memory segment.
- ❖ Other processes that wish to communicate using this shared-memory segment must attach it to their address space.
- ❖ They can then exchange information by reading and writing data in the shared areas.
- ❖ EXAMPLE:PRODUCER – CONSUMER PROCESS:
- ❖ A producer process produces information that is consumed by a consumer process.
- ❖ One solution to the producer–consumer problem uses shared memory
- ❖ To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.

2

- ❖ This buffer will reside in a region of memory that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item.
- ❖ Two types of buffers can be used.
 - o Bounded Buffer.
 - o Unbounded Buffer.
- ❖ The unbounded buffer places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items.
- ❖ The bounded buffer assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.
- ❖ The following variables reside in a region of memory shared by the producer and consumer processes:

```
#define BUFFER SIZE 10
typedef struct {
...

```

```

}item;
item buffer[BUFFER SIZE];
int in = 0;
int out = 0;

```

❖ The variable in points to the next free position in the buffer; out points to the first full position in the buffer. The buffer is empty when $in == out$; the buffer is full when $((in + 1) \% BUFFER\ SIZE) == out$.

❖ CODE FOR PRODUCER PROCESS:

```

item next produced;
while (true) {
/* produce an item in next produced */
while (((in + 1) \% BUFFER SIZE) == out)
; /* do nothing */
buffer[in] = next produced;
in = (in + 1) \% BUFFER SIZE;
}

```

❖ The producer process has local variable next produced in which the new item to be produced is stored.

❖ The consumer process has a local variable next consumed in which the item to be consumed is stored.

❖ This scheme allows at most $BUFFER\ SIZE - 1$ items in the buffer at the same time.

❖ CODE FOR CONSUMER PROCESS

```

item next consumed;
while (true) {
while (in == out)
; /* do nothing */
next consumed = buffer[out];
out = (out + 1) \% BUFFER SIZE;
/* consume the item in next consumed */
}

```

Message-Passing Systems:

- ❖ Message passing provides a mechanism to allow processes to communicate and to synchronize

their actions without sharing the same address space

- ❖ A message-passing facility provides at least two operations:

- o Send(message)

- o Receive(message)

- ❖ If processes P and want to communicate, they must send messages to and receive messages from

each other: a communication link must exist between them.

- ❖ There are several methods for logically implementing a link and the send()/receive() operations:

- o Direct or indirect communication

3

- o Synchronous or asynchronous communication

- o Automatic or explicit buffering

Direct or indirect communication

Direct Communication:

- ❖ Each process that wants to communicate must explicitly name the recipient or sender of the

communication.

- ❖ Direct communication can be done in two ways symmetric addressing and asymmetric

addressing.

- ❖ In Symmetric addressing both the sender process and the receiver process must name the other to

communicate.

- o send(P, message)—Send a message to process P.

- o receive(Q, message)—Receive a message from process Q.

- ❖ In Asymmetric addressing only the sender names the recipient; the recipient is not required to

name the sender.

- o send(P, message)—Send a message to process P.

- o receive(id, message)—Receive a message from any process

❖ A communication link in this scheme has the following properties:

- A link is established automatically between every pair of processes that want to communicate. The processes need to know only each other's identity to communicate.
- A link is associated with exactly two processes.
- Between each pair of processes, there exists exactly one link.

Indirect communication:

❖ With indirect communication, the messages are sent to and received from mailboxes, or ports.

❖ A mailbox can be viewed abstractly as an object into which messages can be placed by processes

and from which messages can be removed. Each mailbox has a unique identification

❖ The send() and receive() primitives are defined as follows:

o send(A, message)—Send a message to mailbox A.

o receive(A, message)—Receive a message from mailbox A.

❖ In this scheme, a communication link has the following properties:

- A link is established between a pair of processes only if both members of the pair have a shared mailbox.
- A link may be associated with more than two processes.
- Between each pair of communicating processes, a number of different links may exist, with each link corresponding to one mailbox.

❖ A mailbox may be owned either by a process or by the operating system.

Synchronous or asynchronous communication

❖ Communication between processes takes place through calls to send() and receive() primitives.

❖ Message passing may be either blocking or nonblocking— also known as synchronous and asynchronous

❖ Blocking send. The sending process is blocked until the message is received by the receiving process or by the mailbox

❖ Nonblocking send. The sending process sends the message and resumes Operation

❖ Blocking receive. The receiver blocks until a message is available.

❖ Nonblocking receive. The receiver retrieves either a valid message or a null.

❖ When both send() and receive() are blocking, we have a rendezvous between the sender and the receiver.

Automatic or explicit buffering:

❖ Messages exchanged by communicating processes reside in a temporary queue.

Basically, such queues can be implemented in three ways:

o Zero capacity.

o Bounded capacity

Unbounded capacity

❖ Zero capacity. The queue has a maximum length of zero. In this case, the sender must block until

the recipient receives the message.

❖ Bounded capacity. The queue has finite length n ; thus, at most n messages can reside in it.

❖ Unbounded capacity. The queue's length is potentially infinite; thus, any number of messages can wait in it. The sender never blocks.

Threads and Processes:

A process has its own system registers and memory stack which helps them in executing threads. Threads are sometimes called lightweight processes.

Threads are very helpful in today's multi-tasking world. Considering MS Word application, it is convenient to highlight the misspelled words while you type; or complete auto-saving of your information as you work on it.

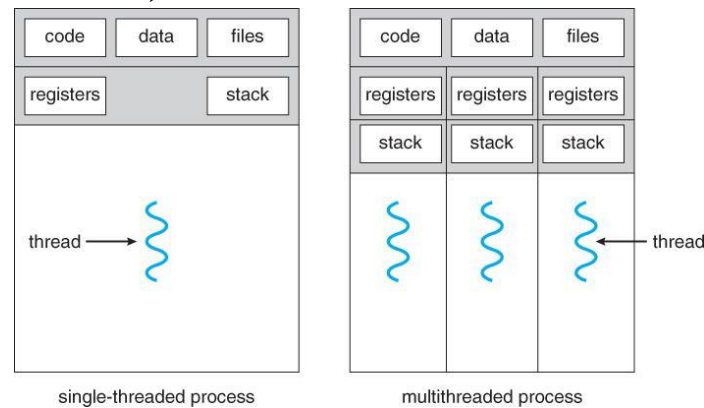
The threads within the application process help you achieve this goal.

Example :

MS Word application, which is a process that runs on computer. But an application can do more than one thing at a time, which means that a given process in an operating system can have one or more threads. Threads represent the actual processing of the code.

In an operating system, a process is a job or a program that can be executed by the computer.

A thread is a basic unit of CPU utilization, consisting of a program counter, a stack, and a set of registers, (and a thread ID.)



Traditional (heavyweight) processes have a single thread of control - There is one program counter, and one sequence of instructions that can be carried out at any given time.

As shown in Figure , multi-threaded applications have multiple threads within a single process, each having their own program counter, stack and set of registers, but sharing common code, data, and certain structures such as open files.

Differences

Processes and thread work together, but they have lots of differences between them. Typically, processes are fairly heavy (like MS Word), while the threads are lighter (like background save option). The table below highlights some of the differences between the two.

Processes	Threads
When switching a process, operating system's resources are required	No OS resources are required for thread-switching
If a process is blocked, other processes waiting in the queue are also blocked	If a thread is blocked, another thread in the same process can still execute
Each process uses same code and has its own memory	All threads can share files and share child processes
An application having multiple processes will use more system resources	Processes using multiple threads use less system resources
Each process works on its own	Threads can access data of other threads

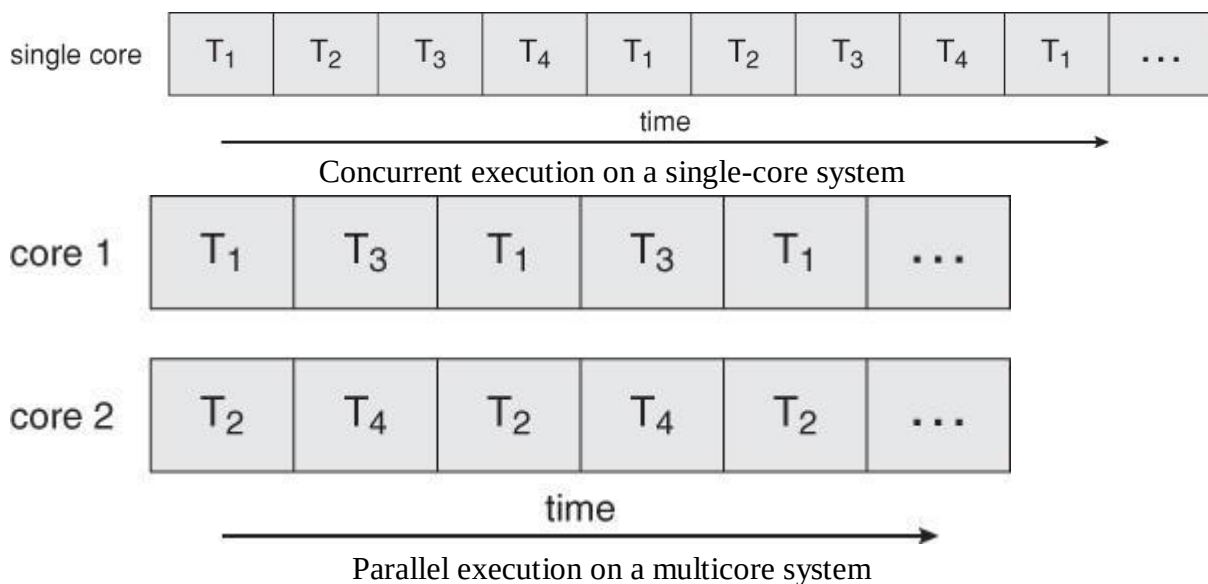
Motivation

- Threads are very useful in modern programming whenever a process has multiple tasks to perform independently of the others.
- This is particularly true when one of the tasks may block, and it is desired to allow the other tasks to proceed without blocking.

- For example in a word processor, a background thread may check spelling and grammar while a foreground thread processes user input (keystrokes), while yet a third thread loads images from the hard drive, and a fourth does periodic automatic backups of the file being edited.
- Another example is a web server - Multiple threads allow for multiple requests to be satisfied simultaneously, without having to service requests sequentially or to fork off separate processes for every incoming request. (The latter is how this sort of thing was done before the concept of threads was developed. A daemon would listen at a port, fork off a child for every incoming request to be processed, and then go back to listening to the port.)

Benefits

- There are four major categories of benefits to multi-threading:
- Responsiveness - One thread may provide rapid response while other threads are blocked or slowed down doing intensive calculations.
- Resource sharing - By default threads share common code, data, and other resources, which allows multiple tasks to be performed simultaneously in a single address space.
- Economy - Creating and managing threads (and context switches between them) is much faster than performing the same tasks for processes.
- Scalability, i.e. Utilization of multiprocessor architectures - A single threaded process can only run on one CPU, no matter how many may be available, whereas the execution of a multi-threaded application may be split amongst available processors. (Note that single threaded processes can still benefit from multi-processor architectures when there are multiple processes contending for the CPU, i.e. when the load average is above some certain threshold.)



- For operating systems, multi-core chips require new scheduling algorithms to make better use of the multiple cores available.
- As multi-threading becomes more pervasive and more important (thousands instead of tens of threads), CPUs have been developed to support more simultaneous threads per core in hardware.

Multi-Threading:

It might be easy to confuse multitasking with multi-threading. Multitasking is a general term for doing many tasks at the same time. On the other hand, multi-threading is the ability of a process to execute multiple threads at the same time.

Again, the MS Word example is appropriate in multi-threading scenarios. The process can check spelling, auto-save, and read files from the hard-drive, all while you are working on a document.

The threads share the same code, files, and data. This means that two or more threads can run at the same time (auto-save, grammar check, spell-check, word-count, etc.).

Multithreading Models

There are two types of threads to be managed in a modern system: User threads and kernel threads.

- User threads are supported above the kernel, without kernel support. These are the threads that application programmers would put into their programs.
- Kernel threads are supported within the kernel of the OS itself. All modern OSes support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.

In a specific implementation, the user threads must be mapped to kernel threads, using one of the following strategies.

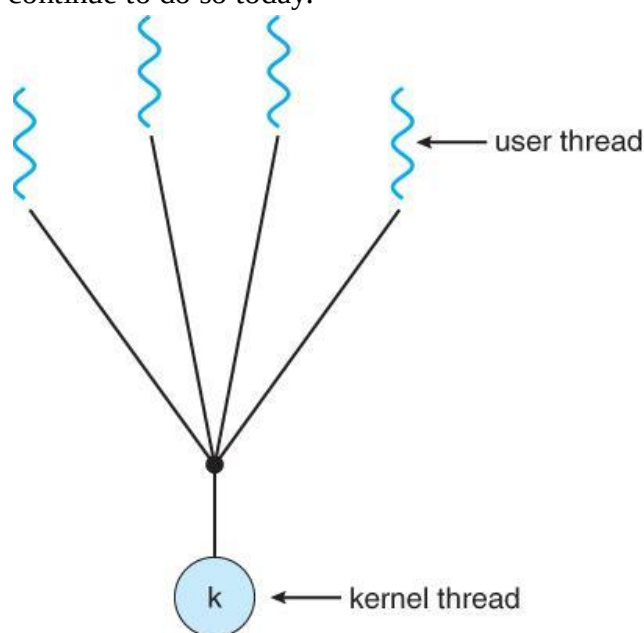
Many-To-One Model

In the many-to-one model, many user-level threads are all mapped onto a single kernel thread. Thread management is handled by the thread library in user space, which is very efficient.

However, if a blocking system call is made, then the entire process blocks, even if the other user threads would otherwise be able to continue.

Because a single kernel thread can operate only on a single CPU, the many-to-one model does not allow individual processes to be split across multiple CPUs.

Green threads for Solaris and GNU Portable Threads implement the many-to-one model in the past, but few systems continue to do so today.



One-To-One Model

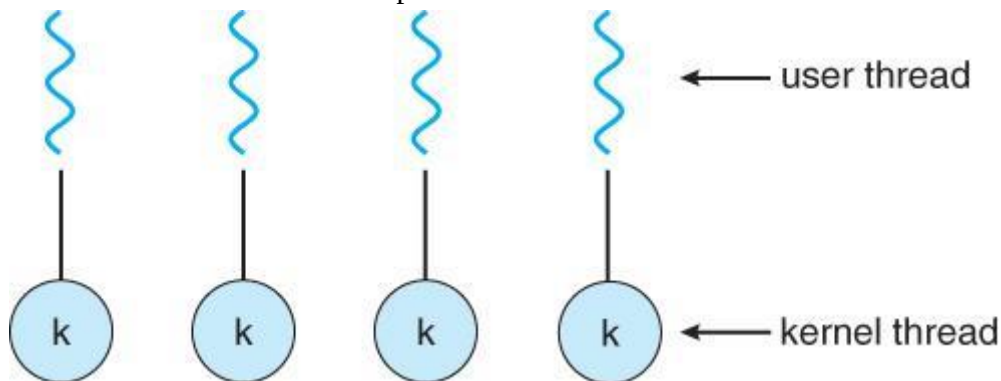
The one-to-one model creates a separate kernel thread to handle each user thread.

One-to-one model overcomes the problems listed above involving blocking system calls and the splitting of processes across multiple CPUs.

However the overhead of managing the one-to-one model is more significant, involving more overhead and slowing down the system.

Most implementations of this model place a limit on how many threads can be created.

Linux and Windows from 95 to XP implement the one-to-one model for threads.



Many-To-Many Model

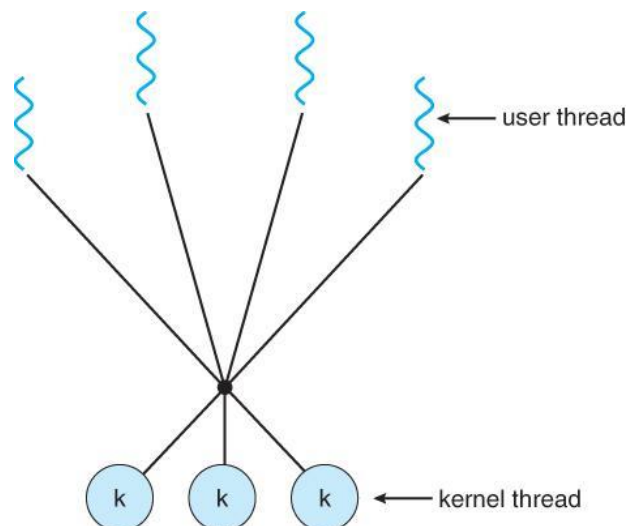
The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one-to-one and many-to-one models.

Users have no restrictions on the number of threads created.

Blocking kernel system calls do not block the entire process.

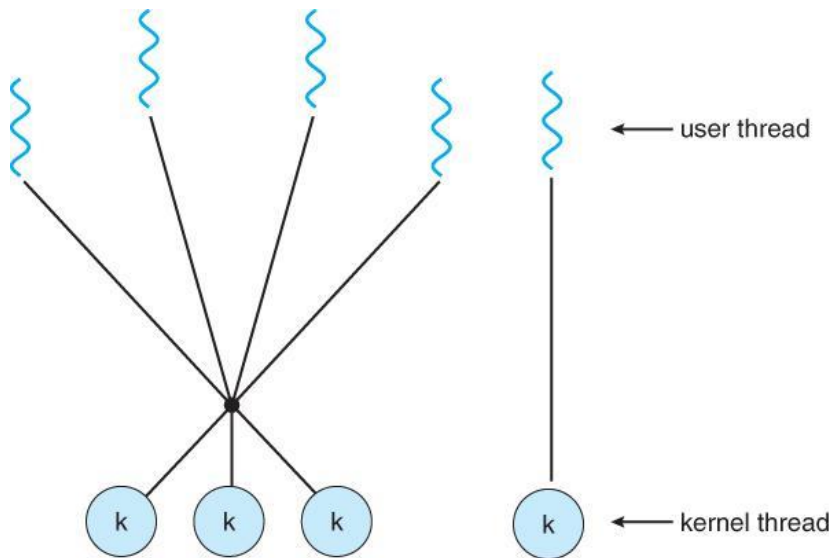
Processes can be split across multiple processors.

Individual processes may be allocated variable numbers of kernel threads, depending on the number of CPUs present and other factors.



One popular variation of the many-to-many model is the two-tier model, which allows either many-to-many or one-to-one operation.

IRIX, HP-UX, and Tru64 UNIX use the two-tier model, as did Solaris prior to Solaris 9.



Thread Libraries

Thread libraries provide programmers with an API for creating and managing threads.

Thread libraries may be implemented either in user space or in kernel space. The former involves API functions implemented solely within user space, with no kernel support. The latter involves system calls, and requires a kernel with thread library support.

There are three main thread libraries in use today:

- POSIX Pthreads - may be provided as either a user or kernel library, as an extension to the POSIX standard.
- Win32 threads - provided as a kernel-level library on Windows systems.
- Java threads - Since Java generally runs on a Java Virtual Machine, the implementation of threads is based upon whatever OS and hardware the JVM is running on, i.e. either Pthreads or Win32 threads depending on the system.

Unit II

CPU SCHEDULING

CPU Scheduling:

Process of allocating CPU to a process in the ready queue. This is done by CPU scheduler. (Short-term scheduler).

Why multiprogramming is needed for CPU scheduling?

- The objective of multiprogramming is maximum **CPU utilization**.
 - In unprocessed System, The memory has only one process. After the completion of process execution CPU sit idle.
 - But **multiprogramming system Memory keeps several jobs**, So We need to keep the CPU as busy as possible. If one process has to wait, the scheduler selects the ready process in memory and allocates it to CPU.

CPU I/O Burst cycle:

- **CPU Burst:** how long a process requires the CPU between I/O waits.
- **I/O Burst:** How long the needs I/O service.

Process execution consists of cycle of CPU execution and I/O wait.

- Process execution begins with a CPU burst that is followed by I/O bursts, then another CPU burst, then another i/o burst and so on.
- The last CPU burst terminates the execution.
- That is depicted in figure.
- The program may be I/O bound or CPU bound.

I/O Bound: This program has very short CPU burst but needs more time to complete I/O.

CPU Bound: This Program has long CPU burst but very short I/O burst.

CPU scheduling decisions may take place under the following four circumstances:

1. When a process switches from the **running state to the waiting state** (for example, I/O request, or invocation of wait for **the termination of one of the child processes**).
2. When a process switches from the **running state to the ready state** (for example, when an interrupt occurs)
3. When a process switches from the **waiting state to the ready state** (for example, completion of I/O).
4. When a **process terminates**.

Non-preemptive scheduling:

- Non-preemptive scheduling, once the CPU has been allocated to a process, **the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state**. This scheduling method is used by the Microsoft Windows 3.1 and by the Apple Macintosh operating systems.
- Non-preemptive scheduling does not require the special hardware.
- In non-preemptive scheduling, a scheduler executes jobs in the following two situations.
 1. When a process switches from running state to the waiting state.
 2. When a process terminates.

Example :FCFS and SJF.

Preemptive scheduling:

In preemptive scheduling, **a process that is being allotted processor is forcibly removed and the processor is given to some other process.**

Situation for Preemption;

1. When a process switches from the running state to the ready state (for example, when an interrupt occurs)
2. When a process switches from the **waiting state to the ready state** (for example, completion of I/O).

Example: Round robin scheduling algorithm, Shortest Remaining Time First Algorithm..

Dispatcher:

The dispatcher is the module that **gives the control of the CPU to the process selected by the short-term scheduler.**

The function of CPU scheduler:

- Switching context
- Switching to the user mode.
- Jumping to the proper location in the user program.

Dispatch latency:

The time take for the dispatcher to stop one process and start another running is known as dispatch latency.

Scheduling criteria:

Scheduling criteria is used to compare various CPU scheduling algorithms. Some criteria's are,

CPU Utilization: Keep the CPU as busy as possible. It may range from 0 to 100 percent.

Throughput:

- The number of process completed per time unit, called throughput.
- For long process it may be 1 process per hour. For short process it may be 10 processes per second.

Turnaround time:

- The interval from the time of submission of a process to the time of completion is the turnaround time.
- Turnaround time is sum of the periods spend waiting to get into memory, waiting in the ready queue, execution on the CPU and doing I/O.

Waiting time: Sum of periods, the process spends waiting in the ready queue.

Response Time:

- Time between submission of requests and first response to the request is called response time.

How you Maximize CPU utilization? To maximize CPU utilization and throughput, we should minimize the turnaround time, waiting time and response time.

SCHEDULING ALGORITHMS:**FIRST COME FIRST SERVED ALGORITHM:**

It is the simplest CPU scheduling algorithm.

- With this scheme, **the process that requests the CPU first is allocated the CPU first.**
- In this algorithm we keep ready queue as a **FIFO queue**, the new process enters the tail of the ready queue.
- When the CPU is free it is allocated to the process at the head of the ready queue. The running process is removed from the ready queue.
- In this scheme the **average waiting time is quite long.**

Example,

If the processes arrive in the order P_1, P_2, P_3 and are served in FCFS order .the length and CPU burst time (ms) of processes are,

Processes	burst time (ms)
P_1	24
P_2	3
P_3	3

Gantt chart:

	P_1	P_2	P_3	
0		24	27	30

Waiting Time:

Process	waiting time
P_1	0
P_2	24
P_3	27

Average waiting time is : $(0+24+27)/3=17\text{ms}$

Turnaround time:

Process	Turn around time (ms)
P_1	24
P_2	27
P_3	30

Average turnaround time : $(24+27+30)/3=71/3=23.6\text{ms}$

Performance of FCFS scheduling:

- The process may be CPU bound or I/O bound.
- The CPU bound process will get the CPU and hold it. During this time all the I/O bound process finish their I/O and move into the ready queue and wait for CPU. While the processes are wait in the ready queue the i/o devices are idle.
- Eventually the CPU bound process finishes its CPU first and moves to an I/O device. During this period all the I/O bound processes finishes its CPU bursts and moves to i/o device. This makes CPU idle and I/O device busy.

Convey effect:

In ready queue, **all the processes are waiting for the big process to get off the CPU.** This effects results in lower CPU and device utilization.

Non preemptive:

The FCFS is non-preemptive. Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by requesting I/O.

SHORTEST JOB FIRST ALGORITHM:

- The scheduling of the process is done on the basis of **process having shortest CPU burst time.**
- When the CPU is available, it is assigned to **the process that has the smallest next CPU burst.**
- If two processes have the same length next CPU burst, **FCFS scheduling is used to break the tie.**
- Note that a more appropriate term would be the shortest next CPU burst, because the scheduling is done by examining the length of the next CPU burst of a process, rather than its total length.
- **SJF is the non-preemptive algorithm.**

Non-preemptive: once CPU given to the process it cannot be preempted until completes its CPU burst.

Example 1:

As an example, consider the following set of processes, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time
P ₁	6
P ₂	8
P ₃	7
P ₄	3

Gantt chart:

P ₄	P ₁	P ₃	P ₂	
0	3	9	16	24

Waiting time:

Waiting time is the sum of the periods spent waiting in the ready queue.

Waiting Time of processes:

Waiting time of P₁: 3

Waiting time of P₂: 16

Waiting time of P₃: 9

Waiting time of P₄: 0

Average waiting Time $(3+16+9+0)/4=7\text{ms}$

Turnaround Time:

The interval from the time of submission of a process to the time of completion is the turnaround time.

Turnaround Time of processes:

Turnaround time of P₁:9

Turnaround time of P₂:24

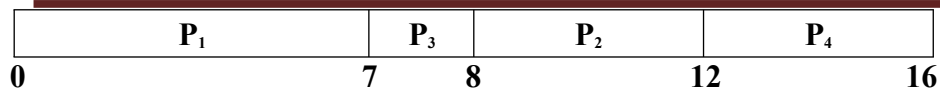
Turnaround time of P₃:16

Turnaround time of P₄:3

Average Turnaround Time $:(9+24+16+3)/4=52/4=13\text{ms}$.

Example2:

Process	Arrival Time	Burst Time
P ₁	0.0	7
P ₂	2.0	4
P ₃	4.0	1
P ₄	5.0	4

Gantt chart:**Waiting Time:**Waiting Time of P₁: 0-0=0Waiting Time of P₂: 8-2=6Waiting Time of P₃: 7-4=3Waiting Time of P₄: 12-5=7

Average waiting time = (0+6+3+7)/4 = 16/4 = 4ms

Turnaround Time:Turnaround Time of P₁: 7-0.0=7Turnaround Time of P₂: 12-2.0=10Turnaround Time of P₃: 8-4.0=4Turnaround Time of P₄: 16-5=11

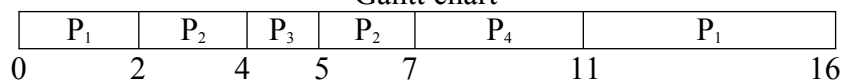
Average Turnaround Time = (7 + 10 + 4 + 11)/4 = 32/4 = 8ms.

Shortest remaining First Algorithm :(SRT)→ It is **preemptive shortest job fist algorithm**.

As an example, consider the following four processes, with the length of the CPU-burst time given in milliseconds:

Process	Arrival time	Burst time
P ₁	0.0	7
P ₂	2.0	4
P ₃	4.0	1
P ₄	5.0	4

Gantt chart

**Waiting Time:**Waiting Time of P₁: (0 -0) + (11-2) = 9Waiting Time of P₂: (2-2) + (5-4) = 1Waiting Time of P₃: (4-4) = 0Waiting Time of P₄: (7-5) = 2

Average waiting time = (9+1+0+2)/4 = 3ms

Turnaround Time:Turnaround Time of P₁: 16-0=16Turnaround Time of P₂: (7-2) = 5Turnaround Time of P₃: 5-4=1Turnaround Time of P₄: 11-4=7**Average Turnaround**

Time = (16+5+1+7)/4 = 29/4 = 7.2ms

- Process P₁ is started at time 0, since it is the only process in the queue. Process P₂ arrives at time 2. The remaining time for process P₁ (5 milliseconds) is larger than the time required by process P₂ (4 milliseconds), so process P₁ is preempted, and process P₂ is scheduled.
- At time 4 the new process P₃ arrived the remaining time for process P₂ (2 milliseconds) is larger than the time required by process P₃ (1 millisecond), so process P₃ is preempted,
- After completion of process P₃, P₄ arrives queue. Now the queue has three processes P₁, P₂, P₄. Then it follows SJF.

Priority scheduling algorithm:

- A priority is associated with each process and the CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS order.
- **High Priority and Low priority:** The CPU is allocated to the process with the highest priority (smallest integer highest priority).
- Priorities are generally some fixed range of numbers, such as 0 to 7, or 0 to 4,095.
- **Priority scheduling algorithm either preemptive or non preemptive.**
- Priorities can be defined either **internally or externally**.
 - **Internally defined priorities** use some measurable quantity or quantities to compute the priority of a process.
 - For example, time limits, memory requirements, the number of open files, and the ratio of average I/O burst to average CPU burst have been used.
 - **External priorities** are set by criteria that are external to the operating system,
 - For example, the importance of the process, the type and amount of funds being paid for computer use, the department sponsoring the work, and other, often political, factors.

Non preemptive scheduling Algorithm:

As an example, consider the following set of processes, assumed to have arrived at time 0, in the order $P_1, P_2, \dots, P_5$, with the length of the CPU-burst time given in milliseconds:

Process	Burst time	priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Gantt chart:

P ₂	P ₅		P ₁		P ₃	P ₄
0	1	6	16	18	23	

Waiting Time:

Waiting time of process $P_1=6$

Waiting time of process $P_2=0$

Waiting time of process $P_3=16$

Waiting time of process $P_4=18$

Waiting time of process $P_5=1$

Average Waiting time = $(6+0+16+18+1) = 41/5 = 8.2\text{ms}$

Turnaround Time:

Turnaround Time of $P_1= 16$

Turnaround Time of $P_2 =1$

Turnaround Time of $P_3=18$

Turnaround Time of $P_4=23$

Turnaround Time of $P_5=6$

Average Turnaround Time= $(16+1+18+23+6)=64/4=16\text{ms}$

PREEMPTIVE PRIORITY SCHEDULING ALGORITHM:

- When a process arrives at the ready queue, its priority is compared with the priority of the currently running process.
- **A preemptive priority-scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.**
- Problem with priority-scheduling algorithms is starvation

Starvation:

A process that is ready to run but lacking the CPU can be considered blocked -waiting for the CPU. **A priority-scheduling algorithm can leave some low-priority processes waiting indefinitely for the CPU.** This is called starvation.

Aging:

Solution for starvation is aging. **Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time.**

Example,

If priorities range from 127 (low) to 0 (high), we could decrement the priority of a waiting process by 1 every 15 minutes. Eventually, even a process with an initial priority of 127 would have the highest priority in the system and would be executed. In fact, it would take no more than 32 hours for a priority 127 process to age to a priority 0 process.

ROUND ROBIN SCHEDULING:

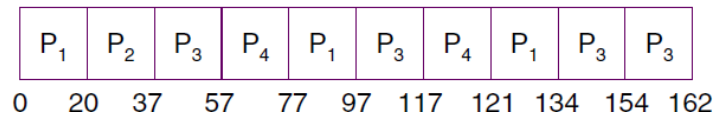
- The round-robin (RR) scheduling algorithm is designed especially **for timesharing systems.**
- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- **It is similar to FCFS scheduling, but preemption is added to switch between processes.**
- **To implement RR scheduling,** we keep the ready queue as a FIFO queue of processes. New processes are added to the tail of the ready queue.
- The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after defined time quantum, scheduler dispatches the process.
- Using this algorithm two things will be happen, they are
 1. **The process may have a CPU burst of less than defined time quantum.**
 - In this case, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue.
 2. **If the CPU burst of the currently running process is longer than defined time quantum,** the timer will go off and will cause an interrupt to the operating system. A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next
- If there are **n processes** in the ready queue and the time **quantum is q ,** then each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
 - **No process waits more than $(n-1) q$ time units.**

Example:

Consider the following set of processes that arrive at time 0ms, with the length of the CPU-burst time given in milliseconds: Example of RR with Time Quantum = 20

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

■ The Gantt chart is:

**Waiting Time:**

Waiting time of process $P_1 = 0 + (77 - 20) + (121 - 97) = 81$

Waiting time of process $P_2 = 20$

Waiting time of process $P_3 = 37 + (97 - 57) + (134 - 117) = 94$

Waiting time of process $P_4 = 57 + (117 - 77) = 97$

Average Waiting Time = $292/4 = 73\text{ms}$

Turnaround Time:

Turnaround Time of $P_1 = 134$

Turnaround Time of $P_2 = 37$

Turnaround Time of $P_3 = 162$

Turnaround Time of $P_4 = 121$

Average turnaround time = $(134 + 37 + 162 + 121)/4 = 113.5\text{ms}$

Explanation:

If we use a time quantum of 20 milliseconds, then process P_1 gets the first 20 milliseconds. Since it requires another 33 milliseconds, it is preempted after the first time quantum, and the CPU is given to the next process in the queue, process P_2 . Since process P_2 does not need 20 milliseconds, it quits before its time quantum expires. The CPU is then given to the next process, process P_3 . Once each process has received its turn, the CPU is returned to process P_1 for an additional time quantum.

Performance of Round Robin scheduling:

The performance of the RR algorithm depends heavily on the size of the time quantum.

→ If the **time quantum is very large** (infinite), the RR policy is the same as the FCFS policy.

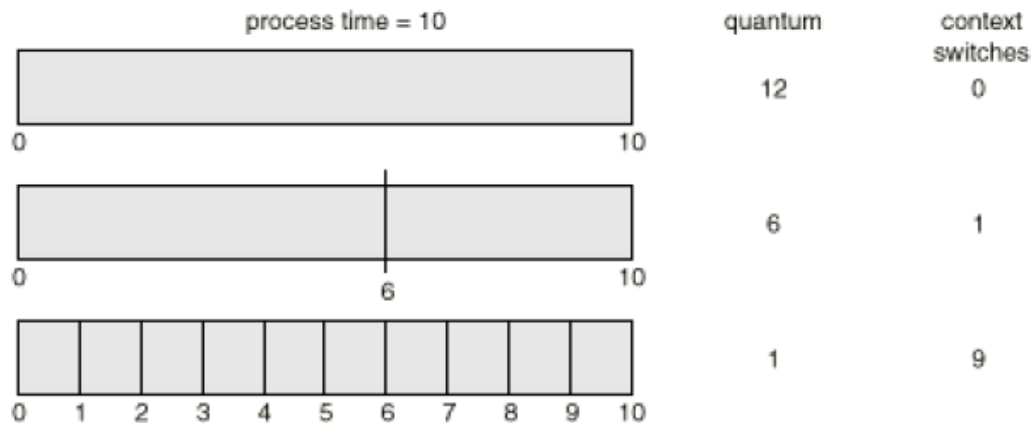
→ If the **time quantum is very small** (say 1 microsecond), the RR approach is called **processor sharing**,

→ **Effect of Context switching:**

- Let us assume that we have only one process of 10 time units. If the quantum is 12 time units, the process finishes in less than 1 time quantum, with no overhead.
- If the quantum is 6 time units, however, the process requires 2 quanta, resulting in 1 context switch.

- If the time quantum is 1 time unit, then 9 context switches will occur, slowing the execution of the process accordingly.
- **Time quantum to be large with respect to the context switch time.** If the context-switch time is approximately 10 percent of the time quantum, then about 10 percent of the CPU time will be spent in context switch.

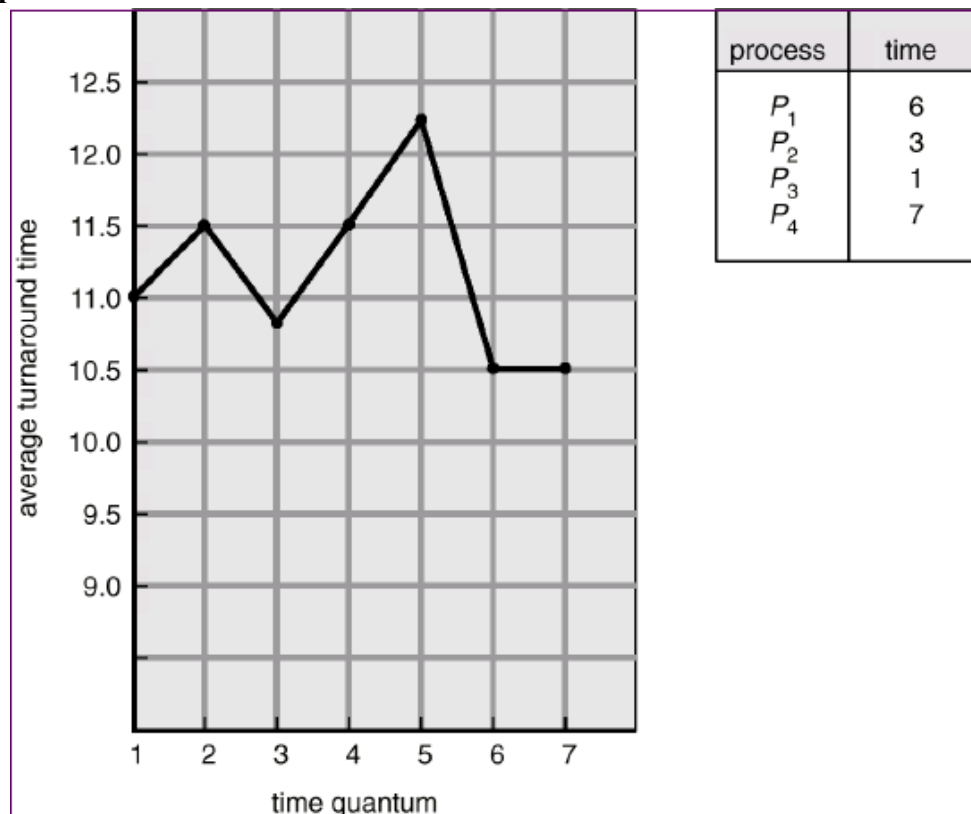
Time quantum and context switch time:



→ Turnaround Time:

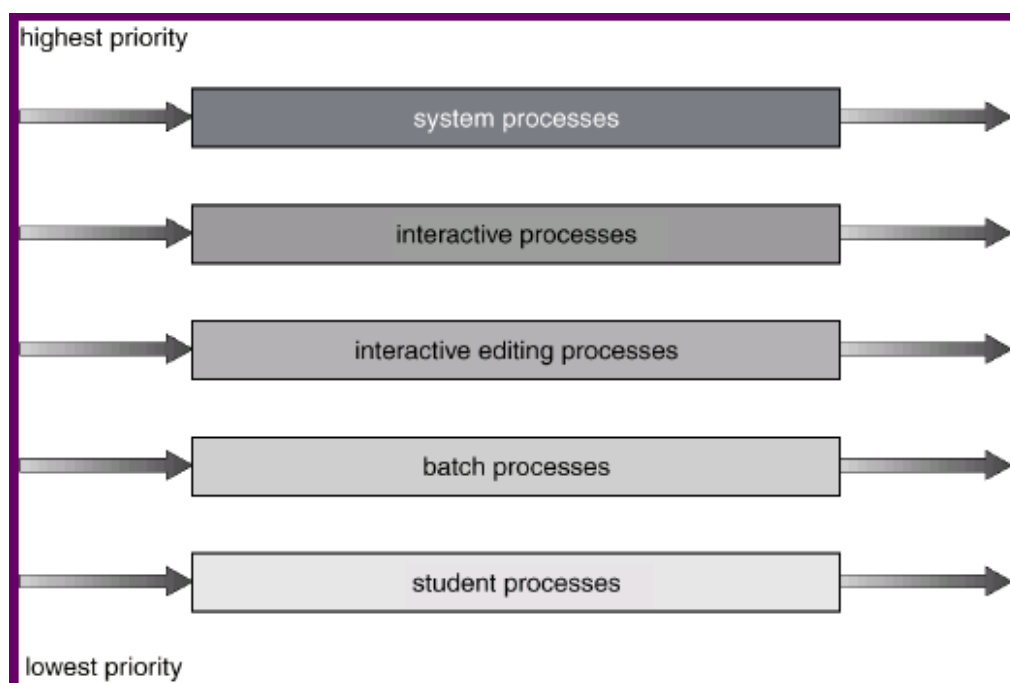
The average turnaround time of a set of processes does not necessarily improve as the time-quantum size increases.

Example



MULTILEVEL QUEUE SCHEDULING ALGORITHM:

- Multilevel queue scheduling is the **extension of priority scheduling**.
- A multilevel queue scheduling algorithm partitions the ready queue in several separate queues, for example,
 - Foreground queues (interactive).
 - Background queues (batch).
 - The foreground processes have higher priority over batch processes.
- In a multilevel queue **scheduling processes are permanently assigned to one queue**.
 - The processes are permanently assigned to one another, based on some property of the process, such as,
 1. Memory size
 2. Process priority
 3. Process type
- Once the processes assigned to the queue, **they cannot change the queue**. i.e processes do not move from one queue to another.
- Algorithm choose the process from the occupied queue that has the highest priority, and run that process either Preemptive or non-preemptively
- **Each queue has its own scheduling algorithm or policy.**
- **Scheduling must be done between the queues**
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR. 20% to background in FCFS

Example:**Multilevel queue scheduling**

Let us look at an example of a multilevel queue-scheduling algorithm with

Five queues:

1. System processes
2. Interactive processes
3. Interactive editing processes
4. Batch processes
5. Student processes

- Each queue has absolute priority over lower-priority queues.
- The batch queue only executes its processes when system process queue, interactive process queue, and interactive process queue were all empty.
- If an interactive editing process entered the ready queue while a batch process was running, the batch process would be preempted. Solaris 2 uses a form of this algorithm.

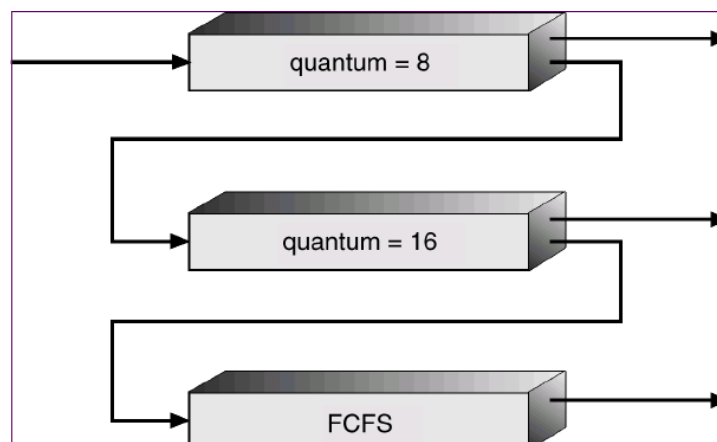
MULTILEVEL FEEDBACK QUEUE SCHEDULING:

- Multilevel feedback queue scheduling algorithm **overcomes problem of multilevel queue scheduling algorithm.**
- Multilevel feedback queue-scheduling algorithm **allows a process to move between queues.** It uses many ready queues and associates a different priority with each queue.
- The Algorithm chooses to process with highest priority from the occupied queue and run that process either preemptively or non-preemptively.
- **If the process uses too much CPU time it will moved to a lower-priority queue.** Similarly, a process that wait too long in the lower-priority queue may be moved to a higher-priority queue. Note that this form of aging prevents starvation.

Multilevel-feedback-queue scheduler defined by the following parameters:

- number of queues
- scheduling algorithms for each queue
- method used to determine when to upgrade a process
- method used to determine when to demote a process
- method used to determine which queue a process will enter when that process needs service

Example:



Three queues:

- ~~Q0 – RR with time quantum 8 milliseconds~~
- Q1 – RR time quantum 16 milliseconds
- Q2 – FCFS Scheduling
- A new job enters queue Q0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q1.
- At Q1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q2.

MULTIPROCESSOR SCHEDULING:

- CPU scheduling **more complex** when multiple CPUs are available.
- **The processors are identical** (or homogeneous) in terms of their functionality; any available processor can then be used to run any processes in the queue. **This system is called homogeneous system.**

Limitations of scheduling in homogeneous processor:

- Consider **a system with an I/O device** attached to a private bus of one processor. Processes wishing to use that device must be scheduled to run on that processor; otherwise the device would not be available.
- **Load sharing**, if several identical processors are available then load sharing can occur. It would be possible to provide a separate queue for each processor. In this case, however, one processor could be idle, with an empty queue, while another processor was very busy. To prevent this situation, we use a common ready queue. All processes go into one queue and are scheduled onto any available processor.

Two Scheduling Approaches:**1. Self scheduling:**

Each processor examines the common ready queue and selects a process to execute. We must ensure that two processors do not choose the same process, and that processes are not lost from the queue.

2. Appointing one processor as scheduler for the other processors, thus creating a master-slave structure.

- The asymmetric multiprocessing is far simpler than symmetric multiprocessing, because only one processor accesses the system data structures, alleviating the need for data sharing.

REAL TIME SCHEDULING:

Real-time computing is divided into two types,

1. Hard real time Systems.
2. Soft real time systems.

HARD REAL TIME SYSTEMS:

Hard real-time systems are required to complete a critical task within a guaranteed amount of time.

Resource reservation:

- Generally, a process is submitted along with a statement of the amount of time in which it needs to complete or perform I/O. The scheduler then either admits the process, guaranteeing that the process will complete on time, or rejects the request as impossible. This is known as resource reservation.
- Such a guarantee is impossible in a system with secondary storage or virtual memory.

SOFT REAL TIME SYSTEMS:

- ~~Soft real-time computing is less restrictive. It requires that critical processes receive priority over less fortunate ones.~~
- Adding soft real-time functionality to a time-sharing system may cause an unfair allocation of resources and may result in longer delays, or even starvation
- So real time system does not support multimedia, high-speed interactive graphics.

Design of scheduler and related aspects:

- **The system must have priority scheduling, and real-time processes must have the highest priority.** The priority of real-time processes must not degrade over time. So disallow process aging on real-time processes, thereby guaranteeing that the priority of the various processes does not change
- The dispatch latency must be small.
 - To keep dispatch latency low, we need to **allow system calls to be preemptible.**
 - To make the entire **kernel preemptible.**
The kernel can always be preemptible, because any kernel data being updated are protected from modification by the high-priority process.

Priority Inversion:

The high-priority process would be waiting for a lower-priority one to finish. This situation is known as priority inversion.

Priority inheritance protocol:

Solution for priority inversion is Priority inheritance protocol. The lower priority process inherits higher priority until they are done with the resource. When it is finished, their priority reverts to its original value.

ALGORITHM EVALUATION:

Selecting a proper scheduling algorithm is a difficult task. Criteria are often defined in terms of CPU utilization, throughput, waiting time or response time.

Methods for evaluating algorithms:

1. Deterministic modeling
2. Queuing model
3. Simulations
4. Implementations

DETERMINISTIC MODELING:

- One major class of evaluation methods is called analytic evaluation.
- **Analytic evaluation** uses the given algorithm and the system workload to produce a formula or number that evaluates the performance of the algorithm for that workload.
- One type of analytic evaluation is **deterministic modeling**. This method takes a particular predetermined workload and defines the performance of each algorithm for that workload.

For example,

Assume that we have the workload shown. All five processes arrive at time 0, in the order given, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time
P ₁	10
P ₂	29
P ₃	3
P ₄	7
P ₅	12

Consider the FCFS,

P ₁	P ₂	P ₃	P ₄	P ₅
0	10	39	42	49
				61

The waiting time is 0 milliseconds for process P₁, 10 milliseconds for process P₂, 39 milliseconds for process P₃, 42 milliseconds for process P₄, and 49 milliseconds for process P₅.

Thus, the average waiting time is $(0 + 10 + 39 + 42 + 49)/5 = 28$ milliseconds.

With nonpreemptive SJF scheduling, we execute the processes as

P ₃	P ₄	P ₁	P ₅	P ₂
0	3	10	20	32
				61

The waiting time is 10 milliseconds for process P₁, 32 milliseconds for process P₂, 0 milliseconds for process P₃, 3 milliseconds for process P₄, and 20 milliseconds for process P₅.

Thus, the average waiting time is $(10 + 32 + 0 + 3 + 20)/5 = 13$ milliseconds.

With the RR algorithm (quantum = 10 milliseconds), we execute the processes as

P ₁	P ₂	P ₃	P ₄	P ₅	P ₂	P ₅	P ₂
0	10	20	23	30	40	50	52
							61

The waiting time is 0 milliseconds for process P₁, 32 milliseconds for process P₂, 20 milliseconds for process P₃, 23 milliseconds for process P₄, and 40 milliseconds for process P₅.

Thus, the average waiting time is $(0 + 32 + 20 + 23 + 40)/5 = 23$ milliseconds.

→ Here SJF policy results in less than one-half the average waiting time obtained with FCFS scheduling; the RR algorithm gives us an intermediate value.

Advantages:

- Deterministic modeling is simple and fast.
- It gives exact numbers, allowing the algorithms to be compared. However, it requires exact numbers for input, and its answers apply to only those cases. T
- The main uses of deterministic modeling are in describing scheduling algorithms and providing examples.

Disadvantages:

Deterministic modeling is too specific, and requires too much exact knowledge, to be useful.

Queueing Models:

- The processes that are run on many systems vary from day to day, so there is no static set of processes (and times) to use for deterministic modeling. What can be determined; however, is the distribution of CPU and I/O bursts. These distributions may be measured and then approximated or simply estimated.
- The result is a mathematical formula describing the probability of a particular CPU burst. Commonly, this distribution is exponential and is described by its mean. Similarly, the distribution of times when processes arrive in the system -the arrival-time distribution- must be given.
- The computer system is described as a network of servers. Each server has a queue of waiting processes. The CPU is a server with its ready queue, as is the I/O system with its device queues. Knowing arrival rates and service rates, we can compute utilization, average queue length, average wait time, and so on. This area of study is called **queueing-network analysis**.

- As an example, let **n** be the average queue length (excluding the process being serviced), let **W** be the average waiting time in the queue, and let λ be the average arrival rate for new processes in the queue (such as three processes per second). Then, we expect that during the time **W** that a process waits, $\lambda \times W$ new processes will arrive in the queue. If the system is in a steady state, then the number of processes leaving the queue must be equal to the number of processes that arrive. Thus,

$$n = \lambda \times W$$

This equation is known as Little's formula.

- Little's formula is particularly useful because it is valid for any scheduling algorithm and arrival distribution. We can use Little's formula to compute one of the three variables, if we know the other two.

For example, if we know that seven processes arrive every second (on average), and that there are normally 14 processes in the queue, then we can compute the average waiting time per process as 2 seconds.

- **Queuing analysis can be useful in comparing scheduling algorithms, but it also has limitations.**

- At the moment, the classes of algorithms and distributions that can be handled are fairly limited.
- The mathematics of complicated algorithms or distributions can be difficult to work with. Thus, arrival and service distributions are often defined in unrealistic, but mathematically tractable, ways.
- It is also generally necessary to make a number of independent assumptions that may not be accurate. Thus, so that they will be able to compute an answer, queuing models are often only **an approximation of a real system. As a result, the accuracy of the computed results may be questionable.**

Simulations:

- To get a more accurate evaluation of scheduling algorithms, we can use simulations. **Simulations involve programming a model of the computer system.** Software data structures represent the major components of the system. The simulator has a variable representing a clock; as this variable's value is increased, **the simulator modifies the system state to reflect the activities of the devices, the processes, and the scheduler.** As the simulation executes, statistics that indicate algorithm performance are gathered and printed.

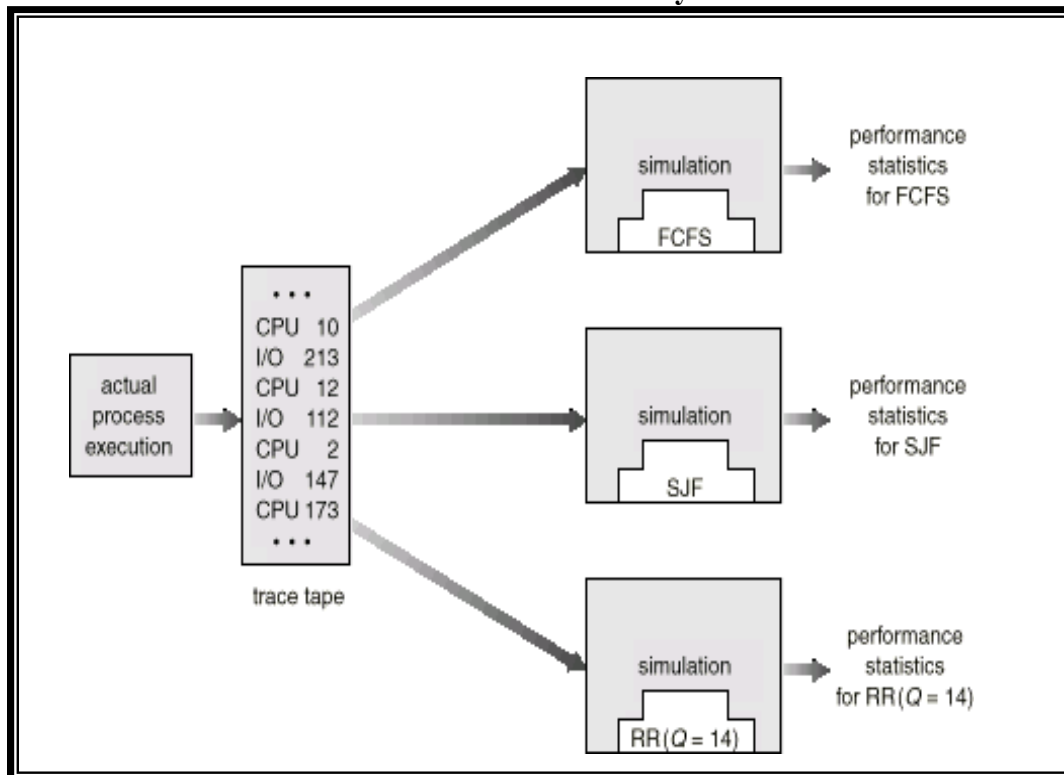
- **The data to drive the simulation can be generated in several ways.**

- The most common method uses a **random-number generator**, which is programmed to generate processes, CPU-burst times, arrivals, departures, and so on, according to probability distributions. The distributions may be defined mathematically (**uniform, exponential, Poisson**) or empirically. If the distribution is to be defined empirically, measurements of the actual system under study are taken.
- The results are used to define the actual distribution of events in the real system, and this distribution can then be used to drive the simulation.
- **A distribution-driven simulation may be inaccurate**, however, due to relationships between successive events in the real system. The frequency distribution indicates only how many of each event occur; it does not indicate anything about the order of their occurrence.

To correct this problem, we can use **trace tapes**. We create a trace tape by monitoring the real system, recording the sequence of actual events. This sequence is then used to drive the simulation.

Trace tapes provide an excellent way to compare two algorithms on exactly the same set of real inputs. This method can produce accurate results for its inputs.

Evaluation of CPU schedulers by simulation



Limitations:

- **Simulations can be expensive**, however, often requiring hours of computer time. A more detailed simulation provides more accurate results, but also requires more computer time.
- In addition, trace tapes can require large amounts of storage space. Finally, the design, coding, and debugging of the simulator can be a major task.

Implementation:

- Even a simulation is of limited accuracy. The only completely accurate way to evaluate a **scheduling algorithm is to code it, put it in the operating system, and see how it works**. This approach puts the actual algorithm in the real system for evaluation under real operating conditions.
- **The major difficulty is the cost of this approach.** The expense is incurred not only in coding the algorithm and modifying the operating system to support it as well as its required data structures, but also in the reaction of the users to a constantly changing operating system. Most users are not interested in building a better operating system; they merely want to get their processes executed and to use their results.
- **The other difficulty with any algorithm evaluation is that the environment in which the algorithm is used will change.** The environment will change not only in the usual way, as new programs are written and the types of problems change, but also as a result of the performance of the scheduler. If short processes are given priority, then users may break larger processes into sets of smaller processes. If interactive processes are given priority over noninteractive processes, then users may switch to interactive use.

Compare FCFS and RR.

Sl no	FCFS	RR
1	FCFS is non preemptive	RR is preemptive
2	Response Time may be high	It Provides good response time
3	It is not useful for time sharing system	It is mainly designed for time sharing system
4	The scheduler allocates the CPU based on process arrival.	It is same like FCFS. The CPU executes the process based on time quantum.

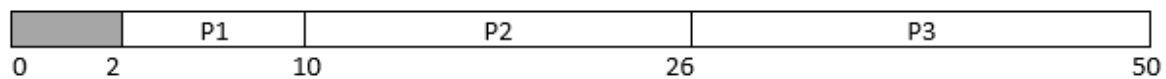
GATE Questions

Q1. Three process P1, P2 and P3 arrive at time zero. The total time spent by the process in the system is 10ms, 20ms, and 30ms respectively. They spent first 20% of their execution time in doing I/O and the rest 80% in CPU processing. What is the percentage utilization of CPU using FCFS scheduling algorithm?

Solution:

Process	Execution time	I/O time	CPU time
P1	10	2	8
P2	20	4	16
P3	30	6	24

Solved Questions based on CPU Scheduling



Since all the processes will first do I/O and then CPU processing, process P1 spends first 20% i.e. 2ms in doing I/O. So CPU is not utilized for the first 2ms. Then P1 spends next 80% i.e. 8ms on processing. By the time P1 finishes with CPU processing P2 has finished its I/O (4ms) and then it gets turn for CPU processing which it does for 16ms and similarly P3 does processing for next 24ms.

Total time = 50ms

CPU utilized for 48ms (starting from 2 and ending at 50)

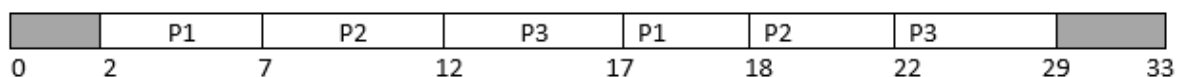
Therefore utilization = $(48/50) \times 100 = 96\%$

Q2. Three process p1, P2 and P3 arrive at time zero. Their total execution time is 10ms, 15ms, and 20ms respectively. They spent first 20% of their execution time in doing I/O, next 60% in CPU processing and the last 20% again doing I/O. For what percentage of time was the CPU free? Use Round robin algorithm with time quantum 5ms.

Solution:

Process	Execution time	I/O burst	CPU burst	I/O Burst
P1	10	2	6	2
P2	15	3	9	3
P3	20	4	12	4

Solved Questions based on CPU Scheduling



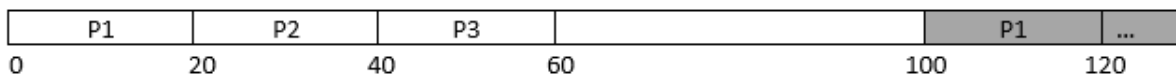
$$\text{Free time} = 2 + 4 [(0-2) + 29-33]$$

$$\text{Unutilized \%age} = (6/33)*100 = 18.18\%$$

Q3. A uniprocessor computer system has three processes, which alternate 20ms CPU bursts with 80ms I/O bursts. All the processes were created at nearly the same time. The I/O of all the processes can proceed in parallel. What will be the CPU utilization (over a long period of time) using FCFS and Round Robin (time quantum 10ms) for this system?

Solution:

FCFS:



Solved Questions based on CPU Scheduling

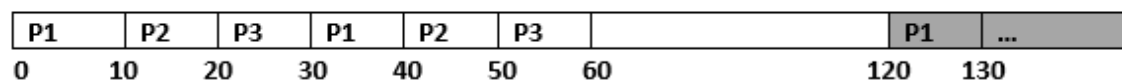
P1 utilizes CPU for 20ms, P2 for next 20ms and P3 for next 20ms. P1 after 20ms goes for I/O for 80ms.

This means it completes I/O at 100ms. After 100ms P1 will again utilize CPU and then P2 after 120ms and so on. This means the CPU remains idle from 60th to 100thms i.e. it is utilized for first 60ms only.

$$\text{Hence CPU utilization} = 60/100 = .60 \text{ i.e. } 60\%$$

RR

RR:



Time slice = 5ms.

P1 utilizes 10ms of CPU and then P2 utilizes 10ms of CPU and the P3 for next 10ms. Hence after 40ms P1 starts with I/O and after 50ms P2 also starts with I/O and P3 after 60ms. Since I/O can be done in parallel, P1 finishes I/O at 120th ms (40 + 80), P2 finishes its I/O at 130th ms (50 + 80) and P3 at 140ms(60+80). Therefore we can see that CPU remains idle from 60th to 120th ms.

That is when Round Robin scheduling is used,

$$\text{Idle time of CPU} = 60\text{ms}$$

$$\text{CPU Utilization} = 60/120 = .50 = 50\%$$

Q4.

Process	Burst Time	Arrival time
P1	4	0
P2	2	1
P3	5	2

Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. Calculate the average waiting time.

Solution:

	P1		P3		P1		P2		P3		P1		P2		P3	
0		2		5	6	7	8	9	10	11						

Waiting time for

$$P1 = 0 + 3 + 2 = 5$$

$$P2 = 2 + 2 + 2 = 6$$

$$P3 = (6 - 1) + 2 = 7$$

$$\text{Hence, Average waiting time} = (5 + 6 + 7) / 3 = 6$$

Unsolved Questions:

Q1. Find the av. waiting time and av. turnaround time using:

- a) FCFS
- b) SJF(NP)
- c) SJF(P)
- d) RR (TQ=3)

Process	Burst Time	Arrival Time
P1	2	2
P2	6	5
P3	4	0
P4	7	0
P5	4	7

Q2. Find the av. waiting time and av. turnaround time using:

- a) SJF(NP)
- b) SJF(P)

PROCESS	BURST TIME	ARRIVAL TIME
P1	3	2
P2	5	0

P3	3	1
P4	4	6

Q3. Find the av. waiting time and av. turnaround time using:

a) Priority(NP)

b) Priority(P)

PROCESS	BURST TIME	ARRIVAL TIME	Priority
P1	3	2	1
P2	5	0	3
P3	3	1	2
P4	4	6	1

Q4. [GATE-2006] Consider three CPU-intensive processes, which require 10, 20 and 30 time units and arrive at times 0, 2 and 6, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end.

(a) 1 (b) 2 (c) 3 (d) 4

Q5. [GATE-2006] Consider three processes (process id 0, 1, 2 respectively) with compute time bursts 2, 4 and 8 time units. All processes arrive at time zero. Consider the longest remaining time first (LRTF) scheduling algorithm. In LRTF ties are broken by giving priority to the process with the lowest process id. The average turn around time is:

(a) 13 units (b) 14 units (c) 15 units (d) 16 units

Q6. [GATE-2007] An operating system uses Shortest Remaining Time first (SRT) process scheduling algorithm. Consider the arrival times and execution times for the following processes:

Process	Execution time	Arrival time
P1	20	0
P2	25	15
P3	10	30
P4	15	45

What is the total waiting time for process P2?

(a) 5 (b) 15 (c) 40 (d) 55

Q7. Consider a system which requires 40-time units of burst time. The Multilevel feedback queue scheduling is used and time quantum is 2 unit for the top queue and is incremented by 5 unit at each level, then in what queue the process will terminate the execution?

Which of the following is false about SJF?

S1: It causes minimum average waiting time

S2: It can cause starvation

(A) Only S1

(B) Only S2

(C) Both S1 and S2

(D) Neither S1 nor S2

Answer (D)

S1 is true SJF will always give minimum average waiting time.

S2 is true SJF can cause starvation.

Q8. Consider the following table of arrival time and burst time for three processes P0, P1 and P2. (GATE-CS-2011)

Process	Arrival time	Burst Time
---------	--------------	------------

P0	0 ms	9 ms
----	------	------

P1	1 ms	4 ms
----	------	------

P2	2 ms	9 ms
----	------	------

The pre-emptive shortest job first scheduling algorithm is used. Scheduling is carried out only at arrival or completion of processes. What is the average waiting time for the three processes?

(A) 5.0 ms

(B) 4.33 ms

(C) 6.33

(D) 7.33

Solution :

Answer: – (A)

Note : Process P0 is allocated processor at 0 ms as there is no other process in the ready queue. P0 is preempted after 1 ms as P1 arrives at 1 ms and burst time for P1 is less than remaining time of P0. P1 runs for 4ms. P2 arrived at 2 ms but P1 continued as burst time of P2 is longer than P1. After P1 completes, P0 is scheduled again as the remaining time for P0 is less than the burst time of P2.

P0 waits for 4 ms, P1 waits for 0 ms and P2 waits for 11 ms. So average waiting time is $(0+4+11)/3 = 5$.

Q9. Consider the following set of processes, with the arrival times and the CPU-burst times given in milliseconds (GATE-CS-2004)

Process	Arrival Time	Burst Time
---------	--------------	------------

P1	0	5
----	---	---

P2	1	3
----	---	---

P3	2	3
----	---	---

P4	4	1
----	---	---

What is the average turnaround time for these processes with the preemptive shortest remaining processing time first (SRPT) algorithm ?

(A) 5.50

(B) 5.75

(C) 6.00

(D) 6.25

Answer (A)

Solution:

The following is Gantt Chart of execution

P1	P2	P4	P3	P1
1	4	5	8	12

Turn Around Time = Completion Time – Arrival Time

Avg Turn Around Time = $(12 + 3 + 6 + 1)/4 = 5.50$

Q10. An operating system uses the Shortest Remaining Time first (SRTF) process scheduling algorithm. Consider the arrival times and execution times for the following processes:

Process	Execution time	Arrival time
---------	----------------	--------------

P1	20	0
----	----	---

P2	25	15
----	----	----

P3	10	30
----	----	----

P4	15	45
----	----	----

What is the total waiting time for process P2?

(A) 5

(B) 15

(C) 40

(D) 55

Answer (B)

At time 0, P1 is the only process, P1 runs for 15 time units.

At time 15, P2 arrives, but P1 has the shortest remaining time. So P1 continues for 5 more time units.

At time 20, P2 is the only process. So it runs for 10 time units

At time 30, P3 is the shortest remaining time process. So it runs for 10 time units

At time 40, P2 runs as it is the only process. P2 runs for 5 time units.

At time 45, P3 arrives, but P2 has the shortest remaining time. So P2 continues for 10 more time units.

P2 completes its execution at time 55

Total waiting time for P2 = Completion time - (Arrival time + Execution time)

= $55 - (15 + 25)$

= 15

